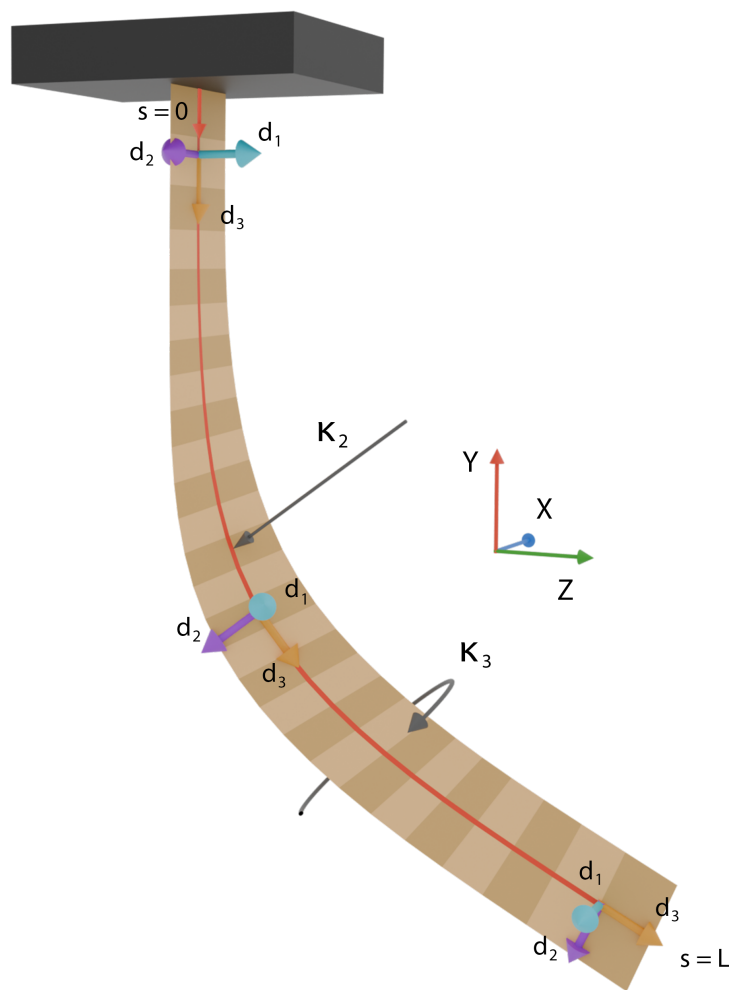




User guide PyElastica

Numerical Simulation: Stresses resulting from ribbon navigation



MAILLARD Pierre

Master student in Mechanical Engineering

LORENZO NOSEDA

June 26, 2025

1 Installation and dependency

All of the source code used to run the simulation is available on this **GitHub** repository: https://github.com/pierremaillard/Ribbon_interaction.git.

The implementation of the model and all the specific functions related to the ribbon model were hard coded directly inside of a merge code between the **PyElastica** source code and a modified version of **PyElastica** by Arman Tekinalp developped for this work: *Topology, dynamics, and control of a muscle-architected soft arm* <https://doi.org/10.1073/pnas.2318769121>).

All of the code architecture is from the version v0.3.2.post1 (April 1, 2024) of **PyElastica** (<https://docs.cosseratrods.org/en/latest/>) with the exception of the object define in the folder `Elastica/Rod`. These objects follows the architecture of an old version of **PyElastica** that Arman Tekinalp used in his work. The main changes bewteen this old version of **PyElastica** and the new one in how the `factory_functions.py` interact with the `memory_blocks.py` functions. Inside the `Rod` folder, beside Arman Tekinalp's implementation of the rods model, two ribbon's model were developed during this project. (`linear_ribbon1D.py` and `ribbon1D.py`)

All of the other functionalities and objects added for this project were build to work inside of the framework of **PyElastica**. It mainly concerns the `_contact_functions.py`, `contact_forces.py` and `external_forces.py` files.

There is no installation procedure except the standard installation of python and the library dependencies. You should be able to run all of the modified library directly form the source code that you can download from the **GitHub** repository. https://github.com/pierremaillard/Ribbon_interaction

1.1 Library Dependencies

This modified version of **PyElastica** requires several library dependencies. The following table outlines the required packages and their purposes:

Package	Version	Purpose
numpy	2.2.6	Core numerical computing and array operations
scipy	1.15.3	Scientific computing functions and algorithms
matplotlib	3.10.3	Basic plotting and visualization capabilities
tqdm	4.67.1	Progress bars for long-running computations
numba	0.61.2	Just-in-time compilation for performance optimization
kaleido	0.2.1	Static image export for Plotly figures
plotly	6.1.1	Interactive plotting and advanced 3D visualizations
pandas	2.2.3	Data manipulation and analysis structures

Table 1: Required dependencies

All dependencies correct version can be installed by running the following line inside of your python environment.

```
pip install -U -r requirements.txt
```


2.2 External python script

Below is a brief overview of the three types of scripts used in this project that are not part of the **Elastica** source code:

- **create_sim_env.py**: This script provides wrapper functions to initialize the simulation environment and configure all components specific to the cases studied in this project.
- **run(...).py**: These Python scripts are executed by the **sbatch** submission scripts to launch simulations on computing clusters. They are responsible for defining the input parameters of each simulation.
- **sim(...).run**: These are **sbatch** scripts used to submit jobs to the cluster. They specify resource allocations (e.g., memory, time, and CPU) and define the paths for storing standard output and error log files.

2.3 Memory blocks and block structures

In **PyElastica** Memory blocks are contiguous memory structures (memory stored in consecutive, adjacent memory addresses with no gaps) that consolidate data from single or multiple rod like object into large, unified arrays. All rods like object are then stored together into block structures. The most significant benefits is memory locality. When computing forces or updating positions across multiple rods, the CPU can access data sequentially from contiguous memory rather than jumping between scatters individual arrays, dramatically improving cache hit rates. This is typically what allows Numba JIT compilation of the vectorized NumPy operation to be this fast.

Here is summary of all the variables (specific to the 1D ribbon class) that are used to describe the 1D ribbon mechanics. All of them are allocate to the memory blocks at the finalization of the system. For more information about the 1D discretizations, refer to **PyElastica** documentation: https://www.cosseratrods.org/cosserat_rods/numerics/

Table 2: Tabulated Summary of Model Variables in PyElastica’s 1D Ribbon Framework

Variable Name	Physical Meaning	Data Type	Units
NODES (N + 1)			
mass	Mass at each node	Scalar	ton
position_collection	3D position coordinates	Vector	mm
velocity_collection	3D velocity components	Vector	m/s
acceleration_collection	3D acceleration components	Vector	m/s ²
internal_forces	Internal forces from the centerline deformation	Vector	N
external_forces	External/applied forces (from BC, contact, damper or interaction modules)	Vector	N
VORONOI (N - 1)			
voronoi_dilatation	Dilatation at Voronoi points	Scalar	—
rest_voronoi_lengths	Reference Voronoi segment lengths	Scalar	mm
kappa	Curvature of the centerline (Cosserat frame)	Vector	1/mm
rest_kappa	Reference curvature (at rest) (Cosserat frame)	Vector	1/mm
internal_couple	Internal bending/twist couple (kappa → constitutive equation)	Vector	N·mm
Continued on next page			

Table 2 – continued from previous page

Variable Name	Physical Meaning	Data Type	Units
ELEMENTS (N)			
width	Cross-sectional width	Scalar	mm
thickness	Cross-sectional thickness	Scalar	mm
volume	Element volume	Scalar	mm ³
density	Material density	Scalar	ton/mm ³
lengths	Current element lengths	Scalar	mm
rest_lengths	Reference/undeformed lengths	Scalar	mm
dilatation	Axial strain measure	Scalar	—
dilatation_rate	Rate of axial strain	Scalar	s ⁻¹
tangents	Element tangent vectors ($\neq d3$)	Vector	unit vector
sigma	Strain on centerline (shear + axial, Cosserat frame)	Vector	—
rest_sigma	Reference strain (at rest) (Cosserat frame)	Vector	—
internal_torques	Internal torques from centerline deformation. All contribution (lagrangian transport + bend/twist + shear/stretch + unsteady dilatation) remapped to element	Vector	N·mm
external_torques	External/applied torques (from BC, contact, etc.)	Vector	N·mm
internal_stress	Internal force from centerline deformation (Cosserat)	Vector	N
omega_collection	Angular velocity (Cosserat frame)	Vector	s ⁻¹
alpha_collection	Angular acceleration (Cosserat frame)	Vector	s ⁻²
director_collection	Cosserat frame (orthonormal basis)	Matrix (3×3)	unit vectors
mass_second_moment_of_inertia	Rotational inertia tensor	Matrix (3×3)	ton·mm ²
inv_mass_second_moment_of_inertia	Inverse rotational inertia tensor	Matrix (3×3)	1/(ton·mm ²)
shear_matrix	Shear stiffness matrix: shear modulus × area	Matrix (3×3)	MPa·mm ²
bend_constants	Bending stiffness matrix: Young's modulus × 2nd moment of area	Matrix (3×3)	MPa·mm ⁴
phi	Evaluated function specific to ribbon model	Scalar	—
phi_p	Derivative of the evaluated function specific to ribbon model	Scalar	—

- **Nodes** ($N + 1$ points): Discrete points along the rod centerline where translational degrees of freedom are defined.
- **Elements** (N segments): Rod segments between consecutive nodes where rotational degrees of freedom, material properties and contact are defined.
- **Voronoi** ($N - 1$ points): Dual mesh points (interior nodes) used for curvature calculations and bending moment computations.

- **Reference frame:** If not specified, all quantities are define with respect to the laboratory frame. (yes even the `internal_forces` array is define on the laboratory frame although it's calculated from an array called `cosserat_internal_stress`)
- **Units:** the units system describe on this table is the unit system used for this project. You can of course adapt it as long as you stay consistent. Note that not all unit systems are usable as you might encounter assertion errors if some values (typically the `mass_second_moment_of_inertia`) are bellow the tolerance of **PyElastica**. ($\approx 10^{-12}$)

2.4 Time stepper logic

PyElastica integrates the equations of motion over time for each rod in the system. At each time step, the internal and external forces and couples acting on the rod are computed at the discrete nodes and Voronoi elements. These forces and torques are then used to update linear and angular accelerations via the balance laws (linear and angular momentum). The resulting accelerations are integrated in time using an explicit or symplectic time-stepping scheme to update the position, orientation, velocity, and angular velocity of the rod.

Internal forces and couples are computed from kinematic quantities using the rod's constitutive relations, which encode the material response to deformation (e.g., stretching, bending, twisting, and shearing). External forces and torques are introduced through modules such as contact, damping, and user-defined forcing, and are updated at each iteration. This dynamic approach allows to easily model contact interactions, as contact forces are recomputed at every time step and directly incorporated into the equations of motion.

2.5 Example: Quasi-static step of ribbon progress inside of an hyper-elastic medium

To illustrate the structure of the code, let's go trough what appends inside of the code when we call the `integrate()` function on a simulation system. (see `Noto_elastica_sleeve_interaction.ipynb` for more details)

In this example the key components of the system are:

- **Ribbon:** the rod-like object. This class main methods are `compute_internal_forces_and_torques(self, time)` and `update_accelerations(self, time)`. The first one update the internal forces and torques using the previous stat geometry and the constitutive law and the second one update the accelerations at each node using the internal forces/torques and the inertial of each element.
- **Damper:** We use the `AnalyticalLinearDamper` class that add damping. This class main method is `dampen_rates()`. It update the velocity block of the ribbon according to an analytic formula.
- **Constrain:** We use the `GeneralConstraint` class at the first and last node of the ribbon. This class main method are `constrain_values()` and `constrain_rates()`. They update the position/velocity and rotation/rotation speed of the first and last node accordingly.
- **Forcing:** We use the `ControlledPushForce` class to apply a force at the base. This class main method is `apply_forces` that comput and add the pushing force to the `external_forces` array on the base node.
- **Contact:** We use the `RibbonSleeveContact` to apply contact between a Sleeve object and the ribbon. This class main method is `apply_contact` that compute and add to the `external_forces` array all of the forces that comes from the interaction between the sleeve and the ribbon.
- **Callback:** We use `SleeveCallBack` and `RibbonCallBack` derived from the `CallBackBaseClass` class to retrieve data at each time step. Their main method is `make_callback` that extract and save the data from the memory block structures.
- **Time Stepper:** We use the `PositionVerlet` class derived from `_SystemCollectionStepper` (define in `symplectic_steppers.py`). This is the core class of the algorithm that will call every other module and integrate the accelerations to advance in time.

The time integration of **PyElastica** is driven by a time-marching scheme implemented in the `integrate()` function (defined in `timestepper/__init__.py`). This function repeatedly calls the `do_step()` method, which performs a single integration step using the specified time stepper (e.g., `PositionVerlet`, `RungeKutta4`, `PEFRL`). Apart from stepping, the `integrate()` function also includes assertion checks and validation at each iteration to detect any numerical instability (e.g., NaNs, which typically signal divergence).

The integration logic is centralized in the `_SystemCollectionStepper` class, specifically in the static method `do_step()`. This method encapsulates the essential sequence of operations that advance the simulation forward. Importantly, it is designed to be agnostic to the specific integration scheme by accepting the kinematic and dynamic operations through the tuple `_steps_and_prefactors`.

Taking the `PositionVerlet` symplectic scheme as an example, the kinematic and dynamic steps are implemented via the functions `overload_operator_kinematic_numba` and `overload_operator_dynamic_numba`, located in `rod/data_structures.py`. These steps are defined as follows:

- The **kinematic step** updates positions and orientations using current velocities:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} + \mathbf{v} \cdot dt, \quad \mathbf{d}_{\text{new}} = \mathbf{R}(\boldsymbol{\omega}, dt) \cdot \mathbf{d}_{\text{old}},$$

where $\mathbf{R}$ is the rotation matrix constructed from the angular velocity.

- The **dynamic step** updates linear and angular velocities:

$$\mathbf{v}_{\text{new}} = \mathbf{v}_{\text{old}} + dt \cdot \frac{d\mathbf{v}}{dt}, \quad \boldsymbol{\omega}_{\text{new}} = \boldsymbol{\omega}_{\text{old}} + dt \cdot \frac{d\boldsymbol{\omega}}{dt}.$$

These updates rely on the forces and torques calculated from the internal constitutive laws (e.g., for a ribbon).

The complete sequence executed by `do_step()` for each integration cycle is as follows:

1. **Kinematic Step:** Positions and orientations are updated.
2. **Apply Value Constraints:** Boundary conditions are imposed on the positions and orientations.
3. **Update Internal Forces:** The method `update_internal_forces_and_torques()` is called for each rod, computing internal elastic responses based on the current configuration.
4. **Synchronize External Inputs:** The `synchronize()` method updates external forces such as contact, damping, and control inputs.
5. **Dynamic Step:** Velocities and angular velocities are updated.
6. **Apply Rate Constraints:** Boundary conditions are imposed on the velocities and angular velocities.
7. **Callback Execution:** User-defined callback functions are triggered to store simulation data.
8. **Reset External Forces:** All external forces and torques are reset to zero in preparation for the next time step. This design simplifies modular force accumulation, as each module (e.g., contact or damping) contributes additively without requiring knowledge of other modules' outputs.

3 Convergence

3.1 Integration Stability

The *PositionVerlet* scheme used in **PyElastica** is an explicit, symplectic integrator, which makes its numerical stability highly sensitive to the choice of time step. Currently, there is no strict convergence rule, but the **Elastica documentation** provides an heuristic:

$$\Delta t \approx \Delta x \sqrt{\frac{\rho}{E}} \quad (1)$$

This estimate is based on the propagation speed of bending and shear waves and ensures that the time step is small enough to resolve these stress wave dynamics. However, in our case, we observe that even for $\Delta t = \frac{50[\text{mm}]}{50} * \sqrt{\frac{1017[\text{kg}/\text{m}^3]}{2.77[\text{GPa}]}} \approx 10^{-7}[\text{s}]$, the simulation remains unstable.

Theoretically, the time step must be sufficiently small to accurately capture the fastest physical phenomena in the system. If the time step is too large, the integration will overshoot and cause numerical instability.

To gain intuition: lighter and smaller elements (with low mass per unit length) require smaller forces for stable integration. In an explicit scheme, forces directly result in accelerations, which are integrated

into velocities and displacements. In this sens, the most critical forces are internal forces resulting from the centerline deformation. This is the reason why stiff systems needs small time step to avoid a catastrophic feed-back loop:

$$\uparrow \text{Internal forces} \rightarrow \uparrow \text{Accelerations} \rightarrow \uparrow \text{Displacements} \rightarrow \uparrow \text{Internal forces} \rightarrow \dots$$

This also explains why simulating ribbons is typically more difficult than rods: the thin thickness results in very light elements, while the large width can still induce significant internal stresses.

3.1.1 Increasing the inertia

Since our primary interest is to obtain the static configuration, a natural strategy to improve numerical stability is to artificially increase the inertia of the system. By increasing inertia, we effectively reduce the magnitude of accelerations induced by internal forces, which leads to more stable time integration.

This approach also addresses another common issue related to floating-point precision. When working with very small inertial values, some computed variables (typically angular momentum in the thin direction) may fall below machine precision (i.e., $< 10^{-12}$ for **Pyelastica**). A typical workaround is to rescale the unit system so that nanometers are of order one ($\text{nm} \sim O(1)$), ensuring all quantities remain within representable bounds. However, in our case, the increased density (by a factor of 100) already elevates the magnitudes of these variables sufficiently to avoid underflow, keeping a rescaling to a more convenient unit system ($\text{mm} \sim O(1)$).

Remark: Another potential stabilization method is to increase the damping in the system. It tends to significantly increase the total real simulation time (stabilization convergence time) without providing as much stability improvement as increasing inertia.

3.2 Quasi-static simulation convergence

To ensure that the system has reached a static equilibrium by the end of the simulation, we monitor the configuration of the ribbon over time and manually adjust the `end_time` parameter accordingly (see `Noto_elastica_ribbon_model.ipynb`). This is a straightforward yet manual approach that relies on visual or numerical inspection. A more robust and automated solution would involve implementing a stopping criterion rather than relying on a fixed-duration `for` loop.

To accelerate convergence to equilibrium, we initialize the ribbon in the configuration of the Sleeve object, which is close to the final static configuration. This significantly reduces transient dynamics. In addition, we tune the damping coefficient to achieve a slightly underdamped response, which helps minimize oscillations while still allowing for rapid dynamic response.

Remark: A principled stopping criterion could involve monitoring the magnitude of the linear and angular velocities (simple approach), or tracking the convergence of the total elastic energy in the system (a more elegant and physically grounded method). Implementing such a criterion would be particularly advantageous in conjunction with an adaptive time-stepping scheme, where velocity norms or energy fluctuations are sometimes already being evaluated. This would allow the simulation to stop when the changes fall below a specified threshold near the level of numerical noise.

4 1D ribbon model

The 1D ribbon model employed in this project is based on a hybrid plate/rod formulation developed by Basile Audoly and Sébastien Neukirch in their work “A one-dimensional model for elastic ribbons: A little stretching makes a big difference” (<https://doi.org/10.1016/j.jmps.2021.104457>). This model captures the full mechanics of an elastic ribbon by reducing the problem to a one-dimensional description along the centerline. All relevant deformations and internal stresses are expressed as functions of the

arc-length coordinate s . It is essentially the same framework as the Kirchhoff rod theory at the exception of the constitutive relation which is derived from the plates theory.

To validate their ribbon model in the static regime, B.Audoly and S.Neukirch employed a fundamentally different numerical approach using **Auto-07P**, a continuation and bifurcation analysis software designed for solving boundary value problems formulated as systems of ordinary differential equations (ODEs). Unlike **PyElastica**, which uses time integration methods to simulate the dynamics of elastic systems, **Auto-07P** focuses on the computation of equilibrium configurations and their stability. By recasting the ribbon model as a boundary value problem, they leveraged **Auto-07P**'s continuation algorithms to track equilibrium branches and detect bifurcations (Buckling).

4.1 Ribbon class

This class is define in `rod/Ribbon1D.py` and is inspired by the **CosseratRod** class from an old version of **PyElastica**. It contains all of the methods used to simulate the mechanical behavior of a ribbon as describe by the one-dimensional model of B.Audoly and S.Neukirch.

Here is a description of the **Ribbon1D** class methods:

- `__init__()`: Initializes a ribbon object with physical, geometrical, and dynamical properties, extending **RodBase** and **KnotTheory**.
- `straight_ribbon()`: Class method to construct a straight ribbon geometry with given material and geometric parameters by calling the `allocate_ribbon()` function.
- `compute_internal_forces_and_torques(time)`: Computes internal elastic forces and torques resulting from shear, stretch, and bending deformations.
- `update_accelerations(time)`: Updates translational and angular accelerations from internal and external forces/torques using Newton-Euler equations.
- `zeroed_out_external_forces_and_torques(time)`: Zeros out external forces and torques; used to reset conditions at each time step.

And here is a description of the associated Numba-accelerated functions:

- `_compute_geometry_from_state(...)`: Computes edge lengths, tangents, and updates local thickness based on volume conservation.
- `_compute_all_dilatations(...)`: Computes both elementwise and Voronoi-region dilatations from current configuration.
- `_compute_dilatation_rate(...)`: Computes the rate of change of dilatation using the positions and velocities of the centerline.
- `_compute_shear_stretch_strains(...)`: Calculates the shear and stretch strain vector **sigma** based on current tangents and directors.
- `_compute_internal_shear_stretch_stresses_from_model(...)`: Computes internal shear and stretch stresses from strain via the shear matrix; enforces geometric constraints numerically.
- `_compute_bending_twist_strains(...)`: Computes curvature tensor κ by applying inverse rotation and scaling by Voronoi rest lengths.
- `_compute_internal_bending_twist_stresses_from_model(...)`: Computes internal bending and twist torques based on current curvature, reference curvature, and nonlinear constitutive laws from Audoly et al.
- `_compute_phi_and_phi_prime(...)`: Computes the nonlinear energy modulation functions $\phi(k_2)$ and $\phi'(k_2)$ using a piecewise approximation from Audoly et al.
- `_compute_internal_forces(...)`: Computes internal force vector from shear and stretch stresses projected to the lab frame and discretized via difference kernels.
- `_compute_internal_torques(...)`: Computes internal torque vector as a combination of bending/twist gradients, shear/stretch couplings, and rotational inertial terms.
- `_update_accelerations(...)`: Updates linear and angular accelerations using internal and external forces and torques, mass, and inertia properties.
- `_zeroed_out_external_forces_and_torques(...)`: Efficiently zeroes out external forces and torques.

4.2 Constitutive law

Adapting the constitutive law from rods to ribbons is relatively straightforward (see `_compute_internal_bending_twist_stresses_from_model()`). However, transitioning from the Cosserat rod framework used in **PyElastica** to the Kirchhoff rod assumptions underlying the 1D ribbon model introduces some limitations. The key distinction is that Cosserat rods allow for shear and stretch deformations of the centerline, whereas Kirchhoff rods (the Audoly–Neukirch ribbon model) assume an inextensible and unshearable centerline.

Since **PyElastica** relies on a time-integration-based solver that requires some compliance in all degrees of freedom, we cannot enforce exact inextensibility or unshearability. To approximate this behavior, we artificially increase the stiffness ($100 \times \text{bending stiffness} \sim \text{shear/stretch stiffness}$) associated with shear and stretch modes. During simulation, we monitor the contribution of these modes to the final static configuration and ensure that their influence remains negligible. This allows us to maintain compatibility with the Kirchhoff-like assumptions of the ribbon model while still using the Cosserat rod formalism required by the solver.

We also need to relax the Audoly–Neukirch ribbon model’s assumption that $k_2 = 0$. Since the stiffness associated with this mode is already much higher compared to k_1 and k_3 , this assumption has a limited to no-effect on the solution. Moreover, relaxing it does not affect the overall derivation of the model (except in the final step where the constitutive equation is obtained) and can therefore be neglected if the constitutive equation is adjusted accordingly.

5 Interaction module: Sleeve object

The interaction forces between the medium and the ribbon is computed using a static ribbon shaped object called "Sleeve". It is define in space by the same system as the ribbon are. (Coordinate of the centerline, Cosserat frame, width, thickness, ...). This static object represent the physical crack created inside of the medium and allows to compute displacement of the medium between it’s rest position (relaxed medium) and the deformed position imposed by the ribbon.

5.1 Sleeve and RibbonSleeveContact classes

The **Sleeve** object inherit of the **SurfaceBase** class and is intended to work with the **RibbonSleeveContact** class which inherit of the **NoContact** class. (used to add interaction between objects) **RibbonSleeveContact** define in `contact_forces.py` uses functions define in `_contact_functions.py` to compute and apply interaction forces. See `Noto_elastica_sleeve_interaction.ipynb` for more details how to use these classes. Here is a description of all the variables used by these two classes.

Table 3: Variables Used in Sleeve and RibbonSleeveContact Classes

Variable Name	Physical Meaning	Data Type	Units
Sleeve Object Variables (N + 1 nodes)			
<code>position_collection</code>	Centerline positions of the sleeve surface	Matrix ($3 \times N+1$)	mm
<code>normal_collection</code>	Surface normal vectors	Matrix ($3 \times N+1$)	unit vector
Sleeve Response Fields (N elements)			
<code>response_force_sleeve</code>	Total contact pressure applied by the medium on the ribbon. This quantity is rescaled to a pressure to be element-size independent	Matrix ($3 \times N$)	MPa
Continued on next page			

Table 3 – continued from previous page

Variable Name	Physical Meaning	Data Type	Units
displacement_sleeve	Relative displacement between ribbon and sleeve at contact points	Matrix ($3 \times N$)	mm
response_couple_sleeve	Total torque per area applied by the medium on the ribbon. This quantity is rescaled by the element area to be element-size independent	Matrix ($3 \times N$)	N/mm
response_couple_local	Local torque in sleeve's frame for twist and bend	Matrix ($3 \times N$)	N·mm
rotation_sleeve	Rotation indicators (sine of twist and bend angles)	Matrix ($3 \times N$)	—
RibbonSleeveContact Parameters			
k	Contact stiffness (spring constant). Here correspond to the shear modulus of the medium	Scalar	MPa
nu	Damping coefficient for normal contact	Scalar	MPa·s
alpha	Ogden model exponent (nonlinear elasticity)	Scalar	—
poisson_ratio	Poisson's ratio for the medium	Scalar	—
N_quad	Number of quadrature points per element (integration resolution for couple computation)	Integer	—
w_grid	Quadrature weights (equal for each point)	Vector (N_quad)	—
base_quad	Quadrature point positions in local element coordinates	Matrix ($1 \times N_quad$)	—
Intermediate Quantities Used in Contact Computation			
normal_displacement_sleeve	Penetration depth of ribbon into sleeve along normal	Vector (N)	mm
hyperelastic_force	Elastic response force from Ogden model	Matrix ($3 \times N$)	N
damping_force	Damping force due to velocity along sleeve normal	Matrix ($3 \times N$)	N
element_position	Ribbon element midpoint positions	Matrix ($3 \times N$)	mm
element_velocity	Ribbon element velocities	Matrix ($3 \times N$)	mm/s
normal_velocity_component	Projection of velocity onto sleeve normal direction	Vector (N)	mm/s
couple_lab	Total contact couple remapped to lab frame	Matrix ($3 \times N$)	N·mm
twist_grid	Local twist positions in element frame	Matrix ($N \times N_quad$)	mm
bend_grid	Local bend positions in element frame	Matrix ($N \times N_quad$)	mm
penetration_twist	Twist rotation angle converted to displacement	Matrix ($N \times N_quad$)	mm
penetration_bend	Bend rotation angle converted to displacement	Matrix ($N \times N_quad$)	mm

Continued on next page

Table 3 – continued from previous page

Variable Name	Physical Meaning	Data Type	Units
<code>forces_twist</code>	Torque due to twist computed from Ogden model	Matrix ($N \times N_quad$)	N
<code>forces_bend</code>	Torque due to bend computed from Ogden model	Matrix ($N \times N_quad$)	N
<code>sin_bend</code>	Sine of bend angle (from cross product)	Vector (N)	—
<code>sin_twist</code>	Sine of twist angle (from cross product)	Vector (N)	—

5.2 Contact force function

limitation and intuition on the force computation

6 Contact Force and Torque Computation Between Ribbon and Sleeve

In this section, we describe the computation of contact interactions between a ribbon object and the surrounding sleeve medium, implemented using hyperelastic force and torque models based on Ogden-type indentation laws. This part of the model is the most limiting in terms of physical interpretation. We advise to carefully validate this model or abandon it for a FEM based relation model.

6.1 Contact Force Model

The contact force model is derived from the Ogden-type response for soft matter indentation, as described in "*Spherical indentation of soft matter beyond the Hertzian regime: numerical and experimental validation of hyperelastic models*", <https://doi.org/10.1007/s10237-008-0139-9>. The normal contact force per element is composed of two parts:

1. A **hyperelastic force**, computed from the indentation depth d of a spherical indenter of radius $R = width/2$ along the ribbon and sleeve normals.
2. A **damping force**, proportional to the normal component of the ribbon velocity to account for energy dissipation. (typically set to zero for this project applications)

The total contact force on each element is thus given by:

$$\mathbf{F}_{\text{contact}} = \mathbf{F}_{\text{hyperelastic}} + \mathbf{F}_{\text{damping}}$$

where

$$\mathbf{F}_{\text{hyperelastic}} = f(d) \cdot \mathbf{n}, \quad \mathbf{F}_{\text{damping}} = -\nu(\mathbf{v} \cdot \mathbf{n})\mathbf{n}$$

Here, $f(d)$ is computed using an Ogden-type expression that includes nonlinear dependence on indentation depth d , ribbon width w , and the Poisson's ratio ν .

$$f(d) = \frac{B\pi a^2}{\alpha} \left[\left(1 - 0.2 \frac{a}{R}\right)^{-\frac{\alpha}{2}-1} - \left(1 - 0.2 \frac{a}{R}\right)^{\alpha-1} \right]$$

$$B = \frac{40E_0}{9\pi(1-\nu^2)}$$

$$a = R^{1/2}d^{1/2}$$

Assumptions:

- The indentation depth is assumed to be in the same order as the ribbon width/2.
- The Ogden model is calibrated using spherical indentation data, which is adapted here to ribbon-medium interaction geometry by interpreting the ribbon width/2 as an equivalent indenter radius.
- Contact occurs only in the normal direction defined by the sleeve's surface.
- The medium is considered homogeneous, isotropic and to follow an Ogden hyperelastic model.

Limitations:

- The use of a scalar projection $d = (\mathbf{X}_{\text{sleeve}} - \mathbf{X}_{\text{ribbon}}) \cdot \mathbf{n}_{\text{sleeve}}$ assumes a fixed normal direction, which may not fully capture complex local deformations.
- The indentation law is design for a spherical shaped indenter.
- Contact is unilateral and frictionless; tangential interactions are not modeled.

6.2 Contact Torque Model

In addition to translational contact forces, the ribbon experiences rotational resistance due to bending and twisting relative to the sleeve. This is modeled by first computing the sine of angular deviations between local material frames:

$$\sin(\theta_{\text{bend}}) = \|\mathbf{d}_1 \times \text{proj}_{\mathbf{d}_1 \mathbf{d}_3}(\mathbf{n}_{\text{sleeve}})\|, \quad \sin(\theta_{\text{twist}}) = \|\mathbf{d}_1 \times \text{proj}_{\mathbf{d}_1 \mathbf{d}_2}(\mathbf{n}_{\text{sleeve}})\|$$

Using these projected angles, a grid-based quadrature is applied to integrate the distributed contact couples across the ribbon width (for twist) and length (for bend). The torque contributions are then computed as:

$$\begin{aligned} \tau_{\text{twist}} &= \int_{-width/2}^{-width/2} f(x\theta_{\text{twist}})x \, dx \\ \tau_{\text{bend}} &= \int_{-L/2}^{-L/2} f(x\theta_{\text{bend}})x \, dx \end{aligned}$$

where $f(x * \theta)$ is the Ogden force model evaluated at a local penetration angle, and x is the spatial coordinate along the corresponding width or length. The penetration depth is assume to be equal to the arc-length of the arc of radius x and angle θ .

The torques are then remapped to the lab frame and added to the external torques on each element.

Assumptions:

- The torque model assumes symmetry in the ribbon cross-section; (underformable elements) hence, twisting and bending resistances are separable.
- Quadrature integration assume that for each penetration depth along the width, all of the assumptions on the contact force model are also valid.

Limitations:

- relying on the arc-sine of projected cross products to compute twist and bend angles means that, for large deformations (e.g., $\theta > 45^\circ$), the computation will leads to ambiguity (wrong computation) and overestimation/underestimations of effective penetration depths.
- Assuming that the force model assumption are true for each position along the with (for quadrature integration) leads to huge approximation of the medium response.

6.3 Note on the FEM based model

We propose to simulate a 2D indentation of the ribbon's cross-section using finite element methods (FEM), with the goal of extracting a more accurate relationship between the indentation depth d , the

twist angle θ_{twist} , the bend angle θ_{bend} , and the resulting force and torque responses. By performing simulations under *pure* and *coupled loading conditions* (e.g., twist + indentation), we can also explore the nonlinear coupling between these modes.

To incorporate this data-driven model into the existing code, only the functions `_calculate_contact_forces_ribbon_sleeve` and `_calculate_contact_torques_ribbon_sleeve` would require modification. The key challenge is to map and validate the response from the 2D cross-sectional model to the 3D behavior of the ribbon element in a way that ensures geometric and mechanical consistency. (Typically, the mechanical behavior of the system should not depend the size of the elements.)

This approach allows to resolve part of the limitation and simplifications of the current theoretical model (e.g., the use of spherical Ogden-type indentation law and the couple reconstruction by integration) and might even speed up the computation. However, it retains the critical assumption that the **local contact 3D deformation** can be approximated by a **2D indentation problem** applied to each ribbon element. This is typically valid only if the ribbon's deformation is smooth along its centerline (if neighboring elements experience similar strain stat). This assumption seems reasonable for small deformation of the medium and small curvature of the centerline but would be hard to validate completely.

7 Working with clusters

The current computation framework is highly CPU-intensive. While Gazzola Lab has noted that GPU parallelization may be feasible in the future, the current CPU implementation is "good-enough" optimized: a complete trajectory analysis can be executed in under 6 hours. Simulations are typically run using 16 CPU cores, which corresponds to half the capacity of a standard node on the JED cluster. Empirical testing shows that allocating more than 16 cores does not yield further speedup due to limited parallel scalability at this stage.

In theory, the total runtime could be reduced to under 20 minutes by parallelizing the independent quasi-static steps across multiple cores or nodes. (simulation of small ribbon at the early stage of the trajectory takes $\sim 15 - 20$ min, for longer ribbon at the end of the trajectory $\sim 5 - 7$ min) The current framework already supports independent execution of each step; implementing basic parallelization would simply require generating a dedicated `sbatch` script to dispatch M independent jobs (where M is the number of steps).

However, this has not been pursued so far, as the current runtime (approximately 8 hours per trajectory) is acceptable, and launching 50 simultaneous jobs across 25 nodes (out of JED's 375 available compute nodes) could be considered inefficient resource usage, especially given the current state of the code. We recommend prioritizing further optimization of the simulation code and improving workload distribution strategies (e.g., more effective data handling and task allocation across cores) before resorting to large-scale job parallelization. Consultation with SCITAS-EPFL staff may also provide valuable guidance on achieving better computational performance.

Remark: The sections of the code most in need of optimization are primarily those developed specifically for this project. Given that the mechanics of the ribbon and its interaction with the `Sleeve` object are inherently more complex than standard rod mechanics or contact with simple geometries (e.g., planes, cylinders, or spheres), slower performance is expected compared to typical `PyElastica` use cases. Nonetheless, we believe that targeted optimization of key functions in the `Ribbon1D` class like `_compute_internal_bending_twist_stresses_from_model(...)` and specific contact functions like `_calculate_contact_forces_ribbon_sleeve(...)` and `_calculate_contact_torques_ribbon_sleeve(...)` could yield significant performance improvements. (specially the quadrature integration computation)

7.1 Quick guide how to use the clusters

This section provides a brief guide on how to access and use the SCITAS clusters for this project. For complete and up-to-date instructions, please refer to the official SCITAS documentation: <https://scitas-doc.epfl.ch/>

We recommend using the **Remote-SSH** extension in Visual Studio Code for convenient interaction with the clusters. Regardless of the method, the following prerequisites must be met:

- A valid **GASPAR** account and approved SCITAS access.
- Connection to the **EPFL network**, either on-campus or via VPN.

Once access is granted, you can connect to the desired cluster (e.g., **jed** or **helvetios**) using the following command in your terminal, replacing `{your_username}` and `{name_of_cluster}` accordingly:

```
ssh {your_username}@{name_of_cluster}.hpc.epfl.ch
```

To initiate a Remote-SSH session in Visual Studio Code:

1. Click the blue button in the bottom-left corner of the VSC interface.
2. Select **Add New SSH Host**.
3. Enter the same SSH command as above.

Create an environment by running:

```
[user@cluster:~]\$ python -m venv --system-site-packages venvs/venv-for-demo
```

Activate it:

```
[user@cluster:~]$ source venvs/venv-for-demo/bin/activate
(venv-for-demo) [user@cluster:~]$
```

Clone the GitHub repository:

```
(venv-for-demo) [user@cluster:~]$ git clone
https://github.com/pierremaillard/Ribbon_interaction.git
```

Install the discrepancies:

```
(venv-for-demo) [user@cluster:~]$ pip install -U -r requirements.txt
```

Start a job:

```
(venv-for-demo) [user@cluster:~]$ sbatch sim_trajectory.run
```

Check if it runs:

```
(venv-for-demo) [user@cluster:~]$ Squeue
(venv-for-demo) [user@cluster:~]$ Sjob {job_id}
```

Retrieving Results to local machine:

From your **local** terminal with SSH access (e.g., **Git Bash** on Windows), you can copy the result folder from the cluster to your local machine using:

```
$ scp -r {your_username}@{name_of_cluster}.hpc.epfl.ch:{path_on_cluster}  
{path_on_local_machine}
```

Example paths:

- path_on_cluster: /home/your_username/Ribbon_interaction/result
- path_on_local_machine: Desktop/Ribbon_interaction_pyelastica/Ribbon_interaction

8 Tips and Tricks

Here is an unstructured list of various *tips and tricks* that may be useful during further development:

- **Interpolation:** To interpolate experimental or simulated data (e.g., FEM results), use the `_bspline` function defined in the `utils.py` file or an adapted version with `spline` instead of `bspline`. You can see the initialization of the class `MuscleTorques` for details how to use it.
- **Basic mathematical functions:** Before implementing any basic mathematical operation, check the `_calculus.py` file—many optimized and vectorized routines are already available there and can prevent performance issues.
- **Navigating the codebase:** Use GitHub’s search functionality directly on the project repository to quickly locate where a function is defined or used. This is particularly helpful for understanding the modular structure and class inheritance in the code.
- **Working with pandas DataFrames:** This project converts simulation outputs (which are usually contiguous dictionaries in **PyElastica**) into pandas DataFrames. While this differs from the typical PyElastica workflow, it greatly simplifies data manipulation and visualization, especially when interfacing with tools like Auto-07P.
- **Debugging with Numba-accelerated functions:** If you need to debug a function compiled with numba header: `@numba.njit(cache=True)` You can comment them out so that the error message makes sens.